

Hierarchical Clustering

G4G + ISLP

The hierarchical clustering has an added advantage over K-means clustering in that it results in an attractive tree-based representation of the observations, called a **dendrogram**.

We are not required to commit to a particular choice of K.

Keyword:

- **Hierarchy:** a system in which groups are ranked above the others.
- **Agglomerative:** collect or form into a large group
- **Singleton pattern:** a software design pattern that ensures a class has only one instance and provides a global point of access to it

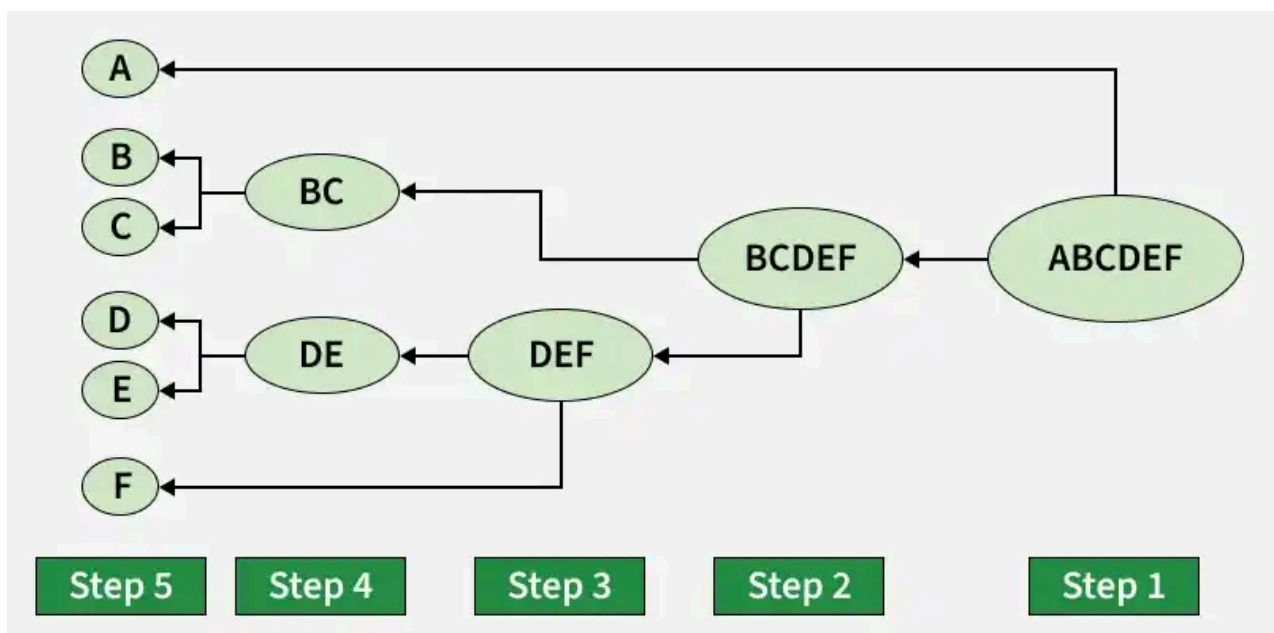
Hierarchical Clustering is an unsupervised learning technique that groups data into a hierarchy of clusters based on similarity. It builds a tree-like structure (dendrogram) that helps visualize relationships and decide the optimal number of clusters.

- Does not require pre-selecting the number of clusters.
- Uses agglomerative or divisive approaches.
- Commonly applied in data exploration and pattern discovery.
- It is commonly used in pattern recognition, customer segmentation, and image grouping.

There are two type of **Hierarchical Clustering**

1. Hierarchical Agglomerative Clustering (Bottom-up approach)

It treats each data as a singleton cluster at the outset and then successively agglomerates pairs of clusters until all clusters have been merged into a single cluster that contains all data.



Workflow

1. **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points, we start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially, since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
6. **Create a dendrogram:** As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram
from sklearn.datasets import make_blobs
```

```
X, _ = make_blobs(n_samples=30, centers=3, cluster_std=10, random_state=42)
```

```
clustering = AgglomerativeClustering(n_clusters=3)
```

```
labels = clustering.fit_predict(X)
```

```
agg = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
```

```
agg.fit(X)
```

```
def plot_dendrogram(model, **kwargs):
```

```
    counts = np.zeros(model.children.shape[0])
```

```
    nsamples = len(model.labels)
```

```
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < nsamples:
                current_count += 1
            else:
                current_count += counts[child_idx - nsamples]
        counts[i] = current_count

    linkage_matrix = np.column_stack(
```

```
[model.children_, model.distances_, counts]).astype(float)
dendrogram(linkage_matrix, **kwargs)
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```
ax1.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=70)
```

```
ax1.set_title("Agglomerative Clustering")
```

```
ax1.set_xlabel("Feature 1")
```

```
ax1.set_ylabel("Feature 2")
```

```
plt.sca(ax2)
```

```
plot_dendrogram(agg, truncate_mode='level', p=5)
```

```
plt.title("Hierarchical Clustering Dendrogram")
```

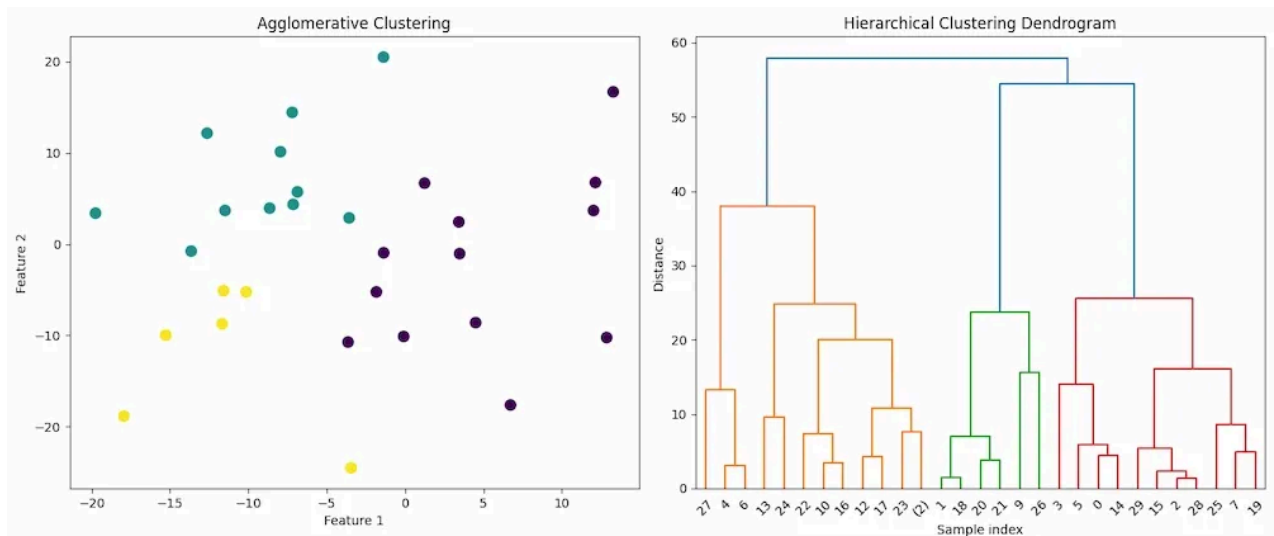
```
plt.xlabel("Sample index")
```

```
plt.ylabel("Distance")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
...
```



2. Hierarchical Divisive clustering (Top-down approach)

It requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data has been split into a singleton cluster.

Workflow

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.

4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from scipy.cluster.hierarchy import dendrogram, linkage

X, _ = make_blobs(n_samples=30, centers=5, cluster_std=10, random_state=42)

def divisive_clustering(data, max_clusters=3):
    clusters = [data]
    while len(clusters) < max_clusters:
        cluster_to_split = max(clusters, key=lambda x: len(x))
        clusters.remove(cluster_to_split)

        kmeans = KMeans(n_clusters=2, random_state=42).fit(cluster_to_split)
        cluster1 = cluster_to_split[kmeans.labels_ == 0]
        cluster2 = cluster_to_split[kmeans.labels_ == 1]

        clusters.extend([cluster1, cluster2])
    return clusters

clusters = divisive_clustering(X, max_clusters=3)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
colors = ['r', 'g', 'b', 'c', 'm', 'y']
for i, cluster in enumerate(clusters):
    plt.scatter(cluster[:, 0], cluster[:, 1], s=50,
                c=colors[i], label=f'Cluster {i+1}')
plt.title('Divisive Clustering Result')
plt.legend()

linked = linkage(X, method='ward')

plt.subplot(1, 2, 2)
dendrogram(linked, orientation='top',
            distance_sort='descending', show_leaf_counts=True)
plt.title('Hierarchical Clustering Dendrogram')

plt.tight_layout()
plt.show()
```

